Εξαμηνιαία εργασία στο μάθημα «Εξόρυξη Δεδομένων και Αλγόριθμοι Μάθησης»

Ακαδημαϊκό έτος 2021-2022

Των φοιτητών: Κοκκάλα Βασιλική(1067519), Φουσκαρής Γιώργος (1067404), έτος 4ο.

Κεφάλαιο 0: Εισαγωγή

Στα πλαίσια του μαθήματος Εξόρυξη Δεδομένων και Αλγόριθμοι Μάθησης κληθήκαμε να υλοποιήσουμε μια πληθώρα συναρτήσεων για την απόκτηση γνώσης από πραγματικά σύνολα δεδομένων. Συγκεκριμένα μας δόθηκαν οι αναλυτικές καταστάσεις παραγωγής και κατανάλωσης ενέργειας για την πολιτεία της Καλιφόρνια μεταξύ 1/1/2019 και 31/12/2021 καθώς και πραγματικά δεδομένα από την βάση δεδομένων της amazon σχετικά με προϊόντα και τις αντίστοιχες αξιολογήσεις τους. Στα επόμενα μέρη θα αναλύσουμε την μέθοδο με την οποία εργαστήκαμε καθώς και τα αποτελέσματα που παρήχθησαν. Ως γλώσσα υλοποίησης επιλέχθηκε η Python . Οι βιβλιοθήκες που χρησιμοποιήθηκαν ήταν οι :sklearn,pandas,matplotlib,numpy,keras

Κεφάλαιο 1: Ενεργειακά Δεδομένα Καλιφόρνιας

Καθαρισμός του dataset

Για την ενοποίηση των δοθέντων αρχείων csv και την ουσιαστική αξιοποίηση τους ήταν αναγκαία η συνένωση των αντίστοιχων αρχείων μεταξύ τους για την παραγωγή ενός συνολικού αρχείου. Ωστόσο κατά την εκτέλεση του κώδικα αντιμετωπίσαμε κάποια σφάλματα. Αυτό συνέβαινε διότι πέραν των πραγματικών ημερών εντός των αρχείων csv υπήρχαν και αρχεία για πλασματικές ημέρες χωρίς φυσικά δεδομένα. Ενδεικτικά ένα τέτοιο αρχείο ήταν το 20190231.csv . Αρχείο που αφορά την 31η Φεβρουαρίου 2019 (!). Για την αγνόηση αυτών των αρχείων χρησιμοποιήσαμε την εξαίρεση EmptyDataError ώστε αυτά τα αρχεία να μην προστεθούν στα τελικά, ενοποιημένα csv.

```
files = os.path.join(path, "20*.csv")
files = glob.glob(files)
files_list = []

#put all the files into a list
for i in range(0, len(files)):
    try:
        pd.read_csv(files[i])
        files_list.append(files[i])
    except EmptyDataError:
        continue
#unify the list into one
df = pd.concat(map(pd.read_csv, files_list), ignore_index=True)
df.to_csv(exported, index=False)  #export to csv
```

Στην συνέχεια παρατηρήσαμε πως αρκετές τιμές παραγωγής ενέργειας ήταν αρνητικές, πράγμα παράλογο για ορισμένες μορφές ενέργειας. Για το υπόλοιπο της εργασίας θα διατηρηθεί η σύμβαση με την οποία εργάστηκε η ομάδα μας. Οι μόνες πηγές ενέργειας οι οποίες δύνανται να έχουν αρνητικές τιμές είναι οι πηγές Large Hydro, Batteries, Imports & Other. Αυτό συμφωνήθηκε λόγω της πιθανότητας υποδομών pumped storage στα μεγάλα υδροηλεκτρικά φράγματα ενώ για τις μπαταρίες οι δυνατότητα φόρτισης τους από το δίκτυο λειτουργεί αντιστοίχως. Και οι δύο αυτές λειτουργίες χρησιμοποιούνται ευρέως στον πραγματικό κόσμο για την ισοστάθμιση παραγωγής και ζήτησης σε στιγμές απότομης μείωσης της ζήτησης. Για τις υπόλοιπες πηγές οι αρνητικές τιμές αρχικά μεταβλήθηκαν σε None και στην συνέχεια με χρήση συναρτήσεων παλινδρόμησης δόθηκαν οι πιθανότερες τιμές βάσει των προηγούμενων και επόμενων τιμών.(forward interpolation)

```
able_to_be_negative_sources = ["Time","Batteries","Large Hydro merged","Imports","Other"]
```

Η στήλη Time δίνεται στον πίνακα μόνο και μόνο για να αγνοηθεί απ την “διόρθωση” των αρνητικών τιμών.

```
#Clean the negative values out of specified sources.
#Fill with none so that the interpolation will fix it later on
for column in df.columns:
   if column not in able_to_be_negative_sources:
     for row in df.index:
       if df.loc[row,column] < 0: df.loc[row,column]= None

fillin_the_missing_values(df)
```

Χρησιμοποιήθηκε παρεμβολή και όχι αντικατάσταση με τη μέση τιμή, καθαρά για λόγους προτίμησης. Επίσης, χρησιμοποιήθηκε προς τα εμπρός και όχι προς τα πίσω διότι

υπήρξαν στήλες οι οποίες ξεκινούσαν χωρίς τιμές και επομένως η προς τα πίσω αντικατάσταση άφηνε το dataset με άδεια κελιά. Σε αντίθεση, με την προς τα εμπρός, η συνάρτηση ψάχνει την πρώτη μη κενή(None) τιμή και την περνά σε όσα κελιά από εκείνο το σημείο και πίσω, δεν έχουν τιμή.

```
# 1.Fill the missing values - interpolation
def fillin_the_missing_values(df):
    # drop_the_duplicates(df)
    df.drop_duplicates(inplace = True)
    #forward interpolation instead of backwards.This way,when all the samples
    #from the start up to a certain point are null,
    #they all shall be filled with the nect following value.
    df.interpolate(method='linear', limit_direction='forward', inplace=True)
```

Επιπροσθέτως αντιμετωπίσαμε το πρόβλημα της λανθασμένης μορφής της ώρας και της καταχώρισης δεδομένων. Επί παραδείγματι, σε ορισμένα αρχεία η ώρα γραφόταν 06:00 ενώ σε λίγα συγκεκριμένα γραφόταν ως 6:00. Αυτό είχε ως αποτέλεσμα την μη αποδοτική παραγωγή δεδομένων και γραφικών συναρτήσεων. Για την αντιμετώπιση του φαινομένου χρησιμοποιήθηκε συνάρτηση η οποία μετέτρεπε την κάθε τιμή στο αναμενόμενο format με δύο ψηφία για την ώρα.

```
#correct wrong time format
df['Time'] = pd.to_datetime(df['Time']).dt.strftime("%H:%M")
```

Ωστόσο λανθασμένη μορφή δεδομένων είχαμε και στις στήλες Large Hydro, Large hydro και Natural Gas, Natural gas για τις οποίες σε συγκεκριμένα στοιχεία τα δεδομένα καταχωρούνταν στις στήλες με τα μικρά γράμματα.Για την ενοποίηση των στηλών οι τιμές προστέθηκαν μεταξύ τους και καταχωρήθηκαν σε νέα στήλη που χαρακτηρίστηκε merged, καταργώντας τις δύο προηγούμενες. Προέκυψαν λοιπόν οι στήλες Natural Gas merged και Large Hydro merged. Είχε προηγηθεί μελέτη των γραφικών παραστάσεων ώστε να βεβαιωθούμε ότι δεν υπήρχε αλληλοεπικάλυψη ή μεγάλη απόκλιση των τιμών στο σημείο συνένωσης τους.

```
#if we're cleaning the sources, merge the four broken columns and the negative values- wrong values
if type == 0:
 #merge seperated columns
 df['Natural Gas merged'] = df['Natural Gas'].fillna(0) + df['Natural gas'].fillna(0)
 df['Large Hydro merged'] = df['Large Hydro'].fillna(0) + df['Large hydro'].fillna(0)

 #drop function which is used in removing or deleting rows or columns from the CSV files
 df.drop('Natural gas', inplace=True, axis=1)
 df.drop('Natural Gas', inplace=True, axis=1)
 df.drop('Large Hydro', inplace=True, axis=1)
 df.drop('Large hydro', inplace=True, axis=1)
```

(Η εντολή if παραπέμπει στην είσοδο type κατά την κλήση της συνάρτησης η οποία δηλώνει εάν καθαρίζουμε το αρχείων των πηγών ή της ζήτησης.)

Τέλος, καλούμε τις συναρτήσεις για τον “καθαρισμό” των αρχείων και τις αποθηκεύουμε σε μεταβλητές τύπου dataframe εφόσον. η clean_csv επιστρέφει το καθαρισμένο dataframe αφότου το κάνει export .

```
    df.to_csv(exported, index=False)  # to eniaio csv file

    return df
```

```
df_sources = clean_csv(sources, sources_exported_file,0)
df_demand = clean_csv(demand, demand_exported_file,1)
```

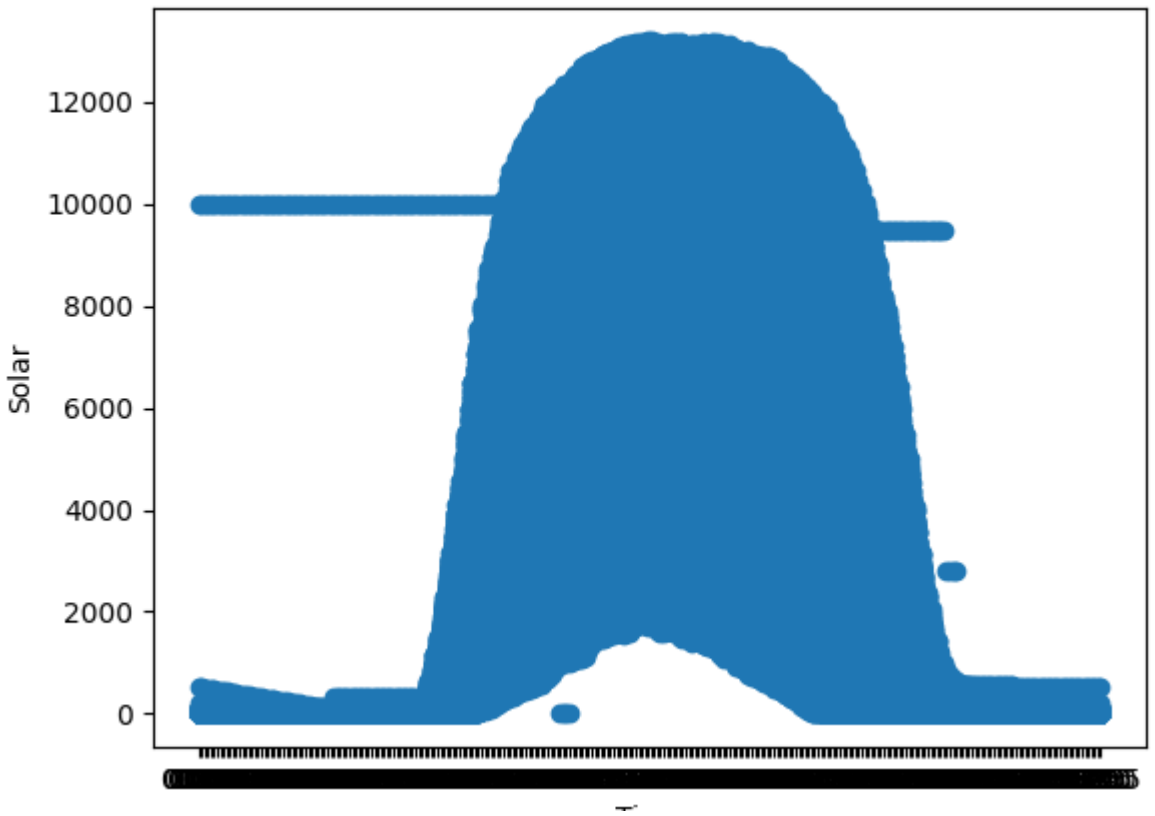
Γραφική Αναπαράσταση των δεδομένων και βασικά στατιστικά στοιχεία

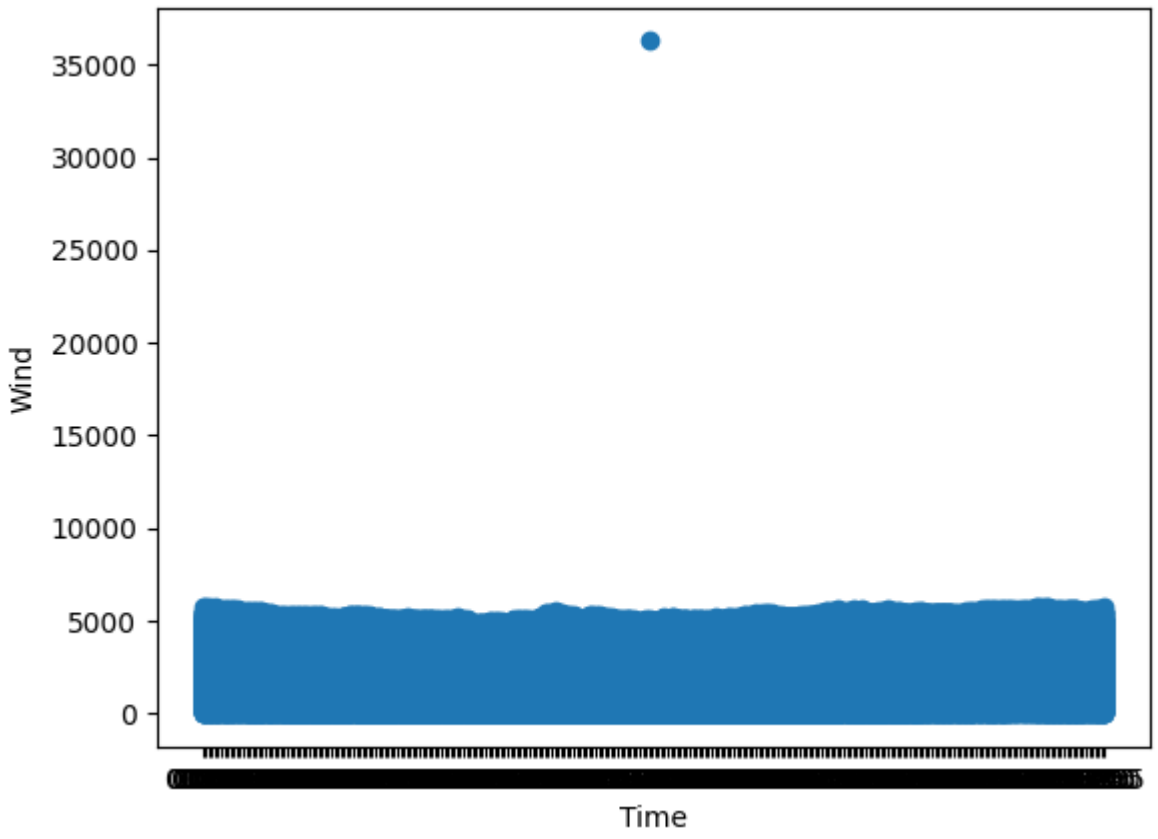
Για την οπτικοποίηση των διαθέσιμων δεδομένων χρησιμοποιήθηκαν δύο τύποι γραφημάτων, ένα διάγραμμα Scatter που συγκέντρωνε τις τιμές ως προς την ώρα της ημέρας, καθώς και ένα διάγραμμα της εκάστοτε μεταβλητής ως προς την χρονική στιγμή.

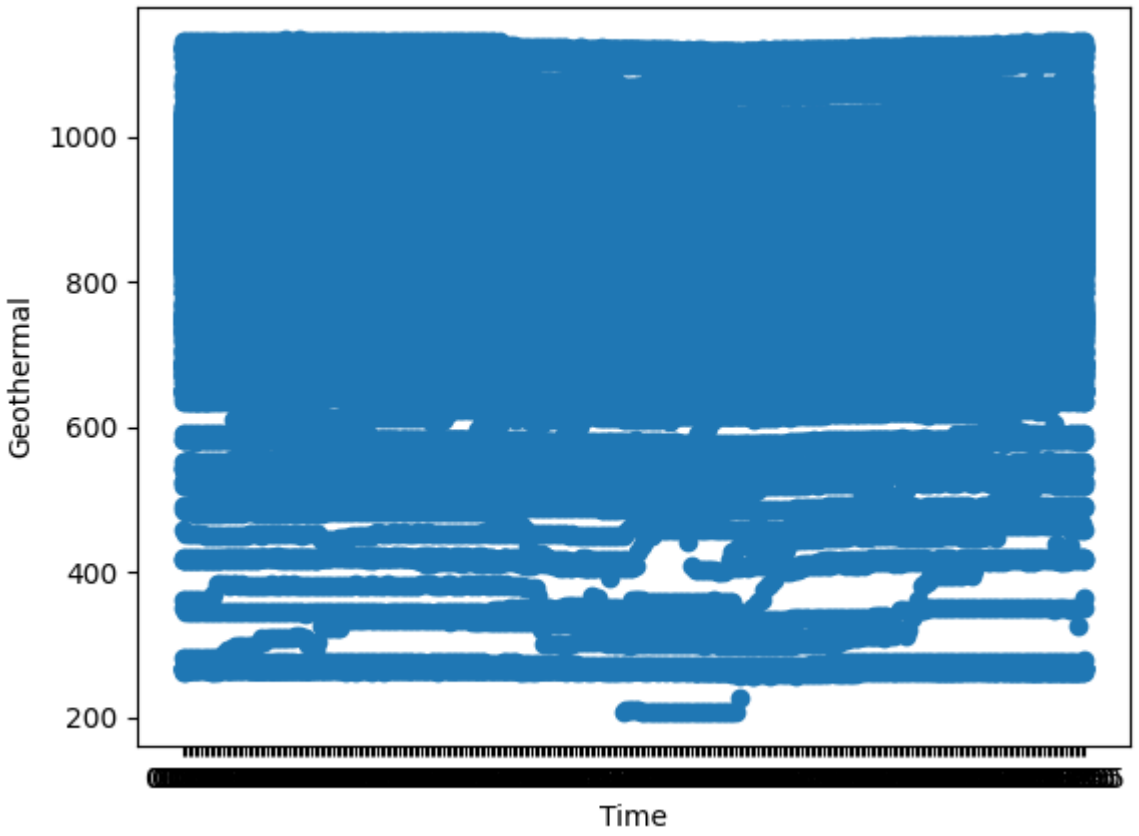
Παρατίθενται οι δύο γραφικές αναπαραστάσεις με σχόλια καθώς και τα ανάλογα στατιστικά δεδομένα:

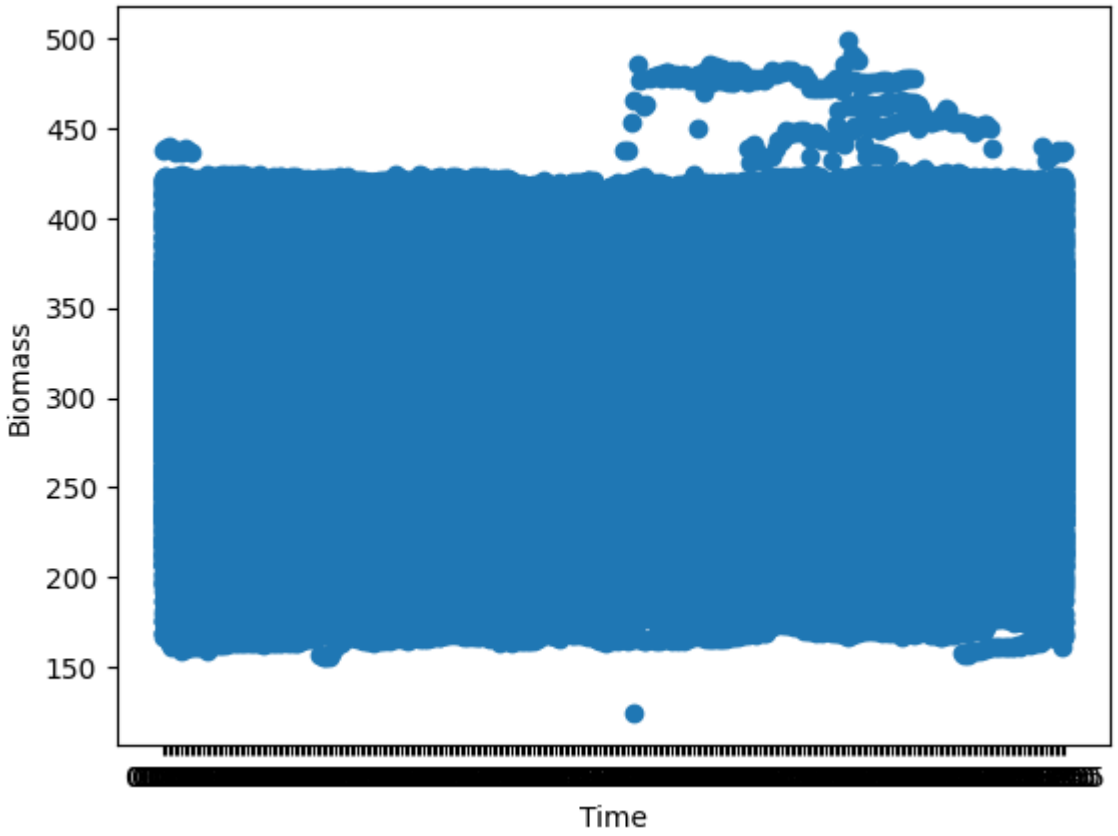
(Οι γραφικές περιέχουν τις τιμές outliers)

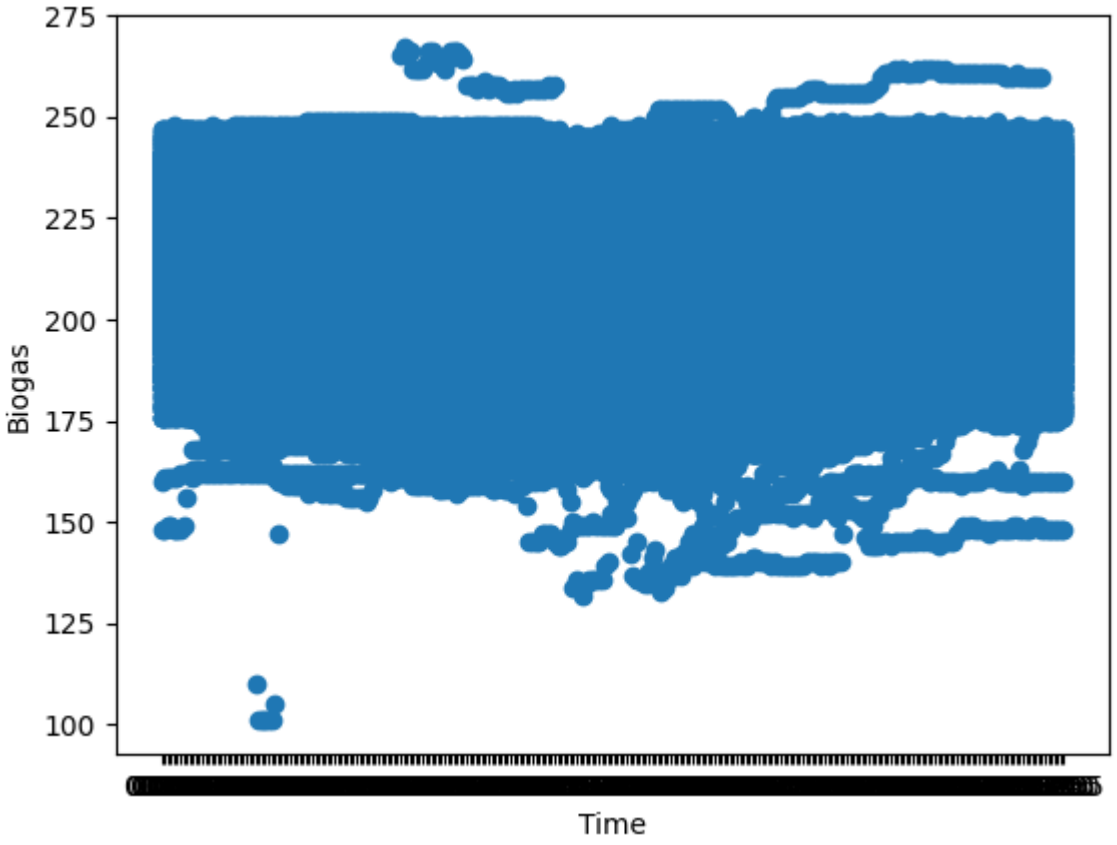
```
plt.scatter(df_sources.Time,df_sources.Solar)
plt.xlabel("Time")
plt.ylabel("Solar")
plt.show()
```

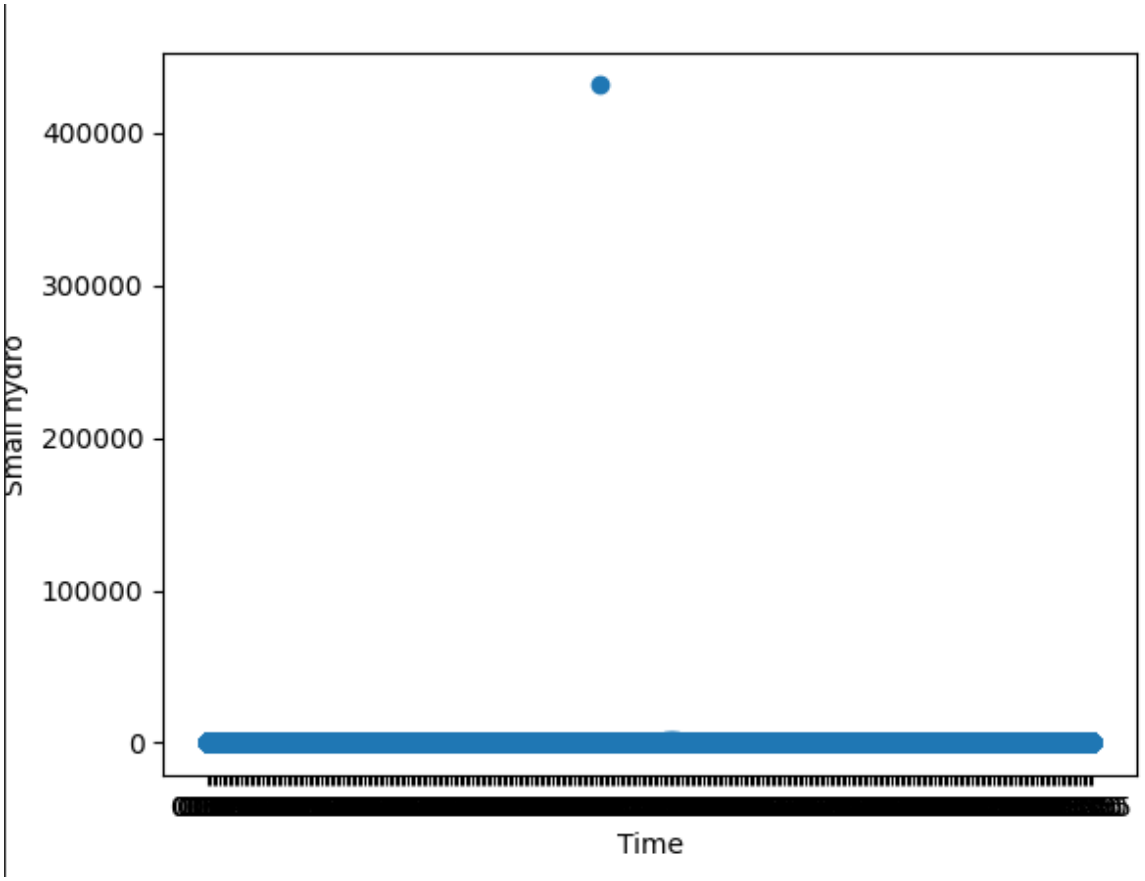

12000
10000
8000
6000
4000
2000
0
Solar

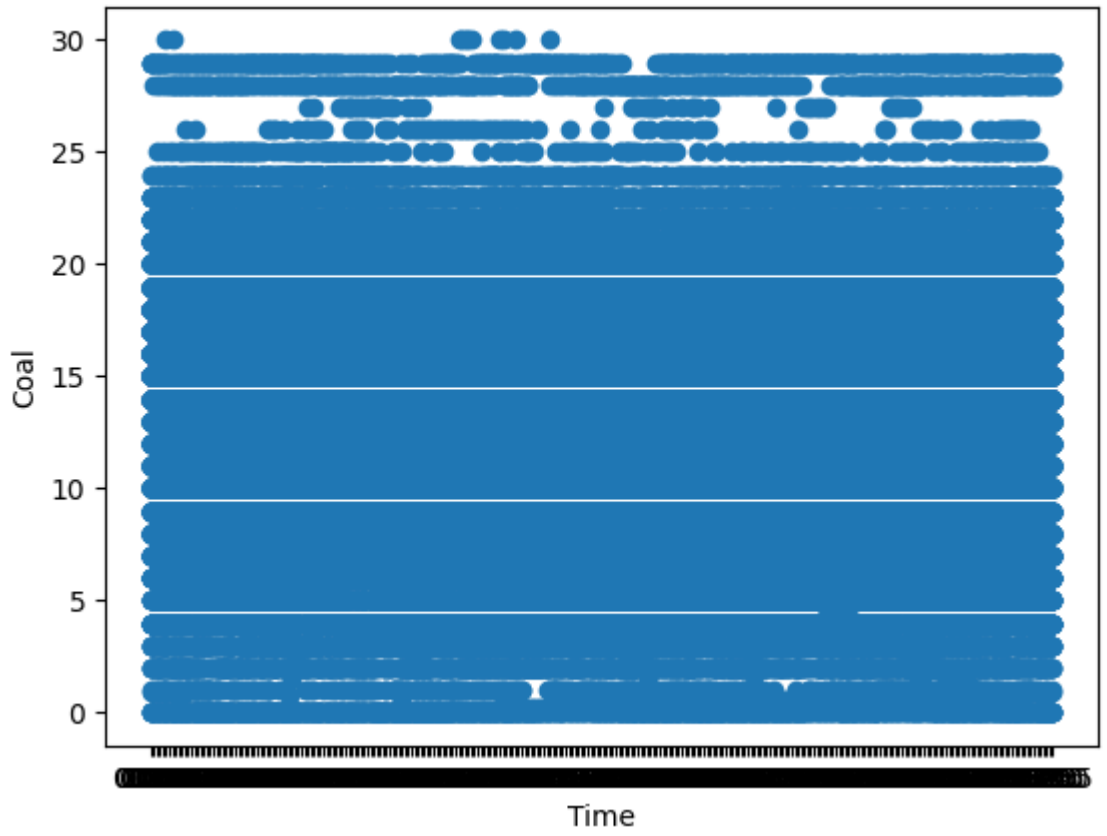

35000
30000
25000
20000
15000
10000
5000
0
Wind
Time

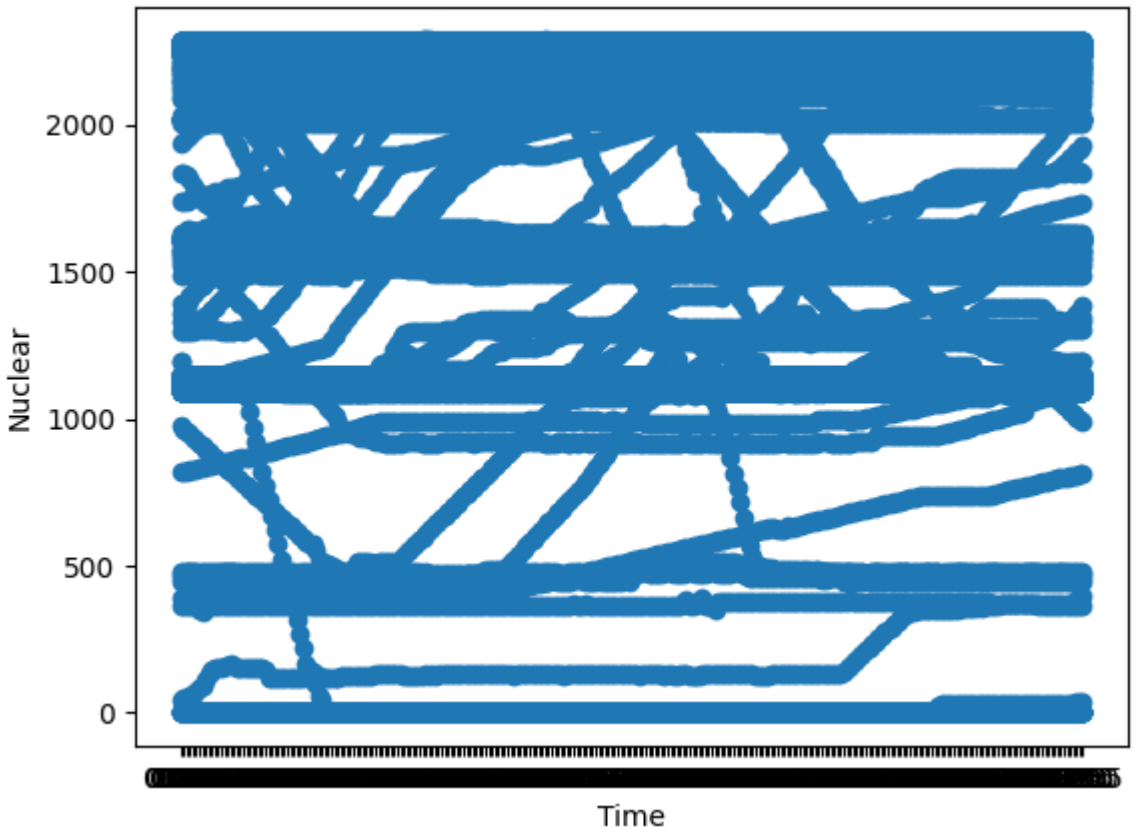

Geothermal
1000
800
600
400
200
Time

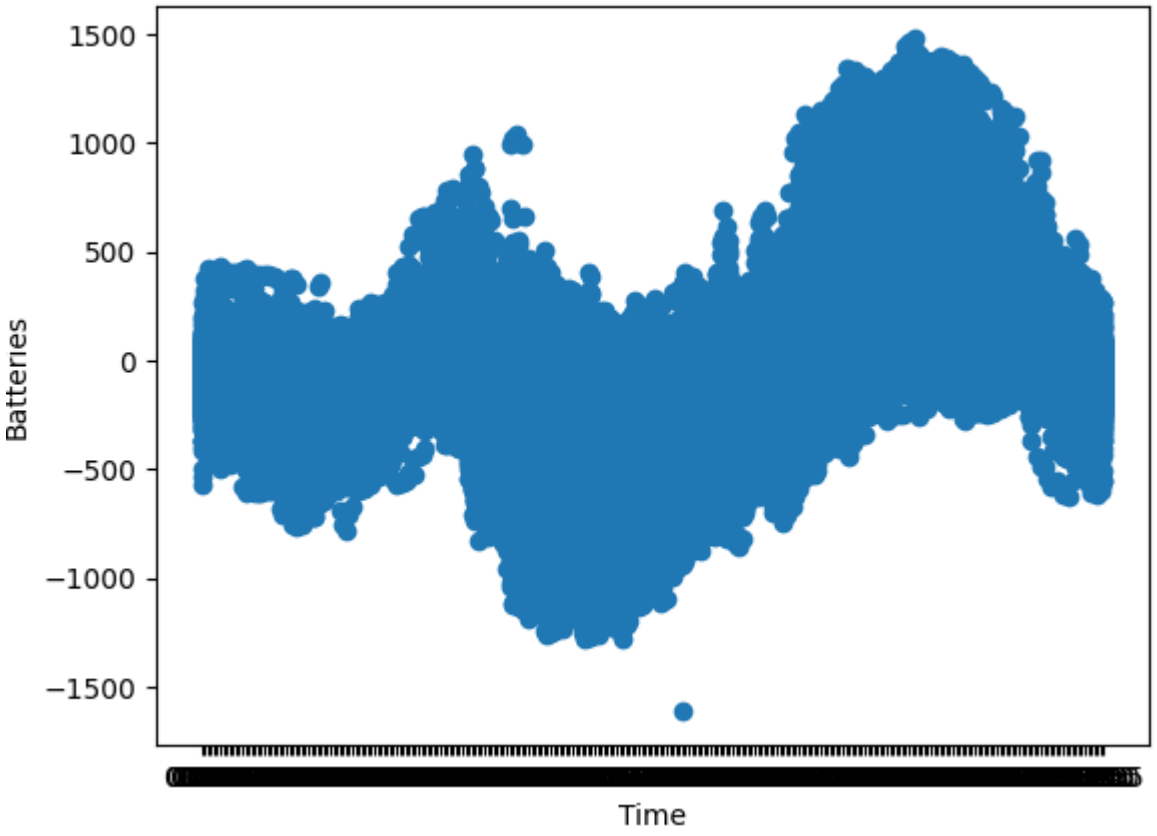

Biomass
500
450
400
350
300
250
200
150
Time

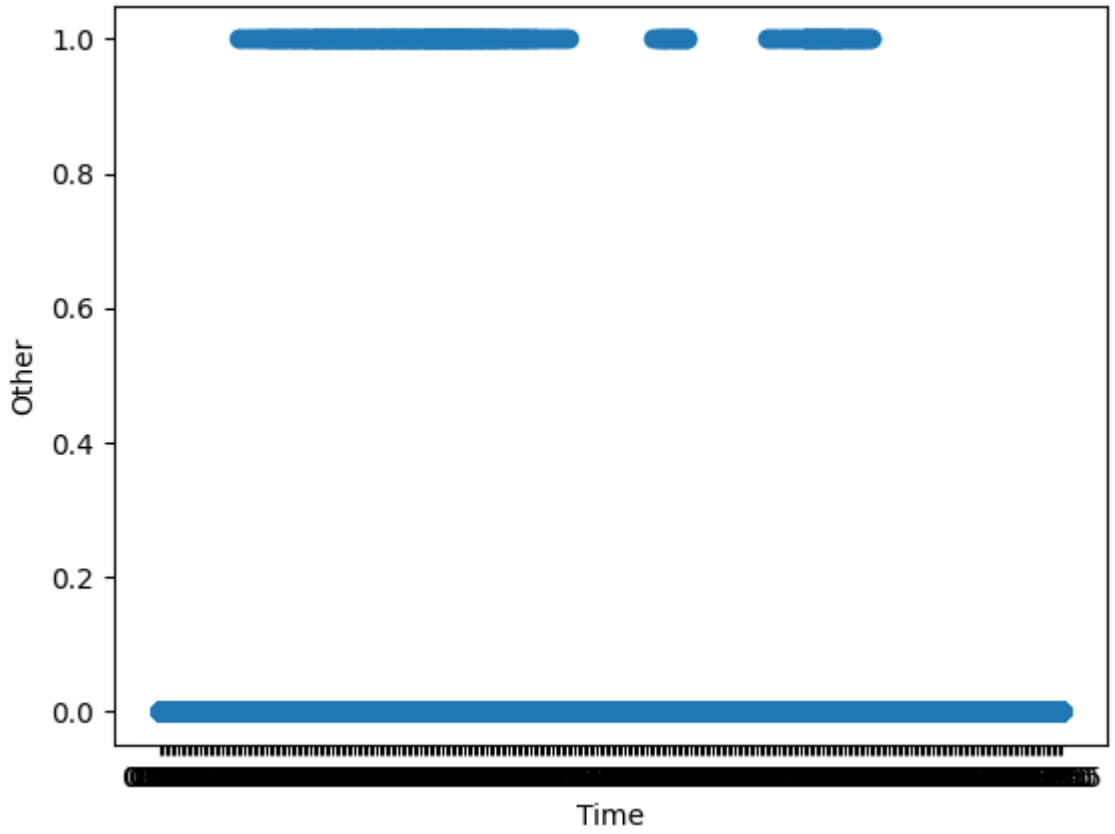

275
250
225
200
175
150
125
100
Biogas
Time

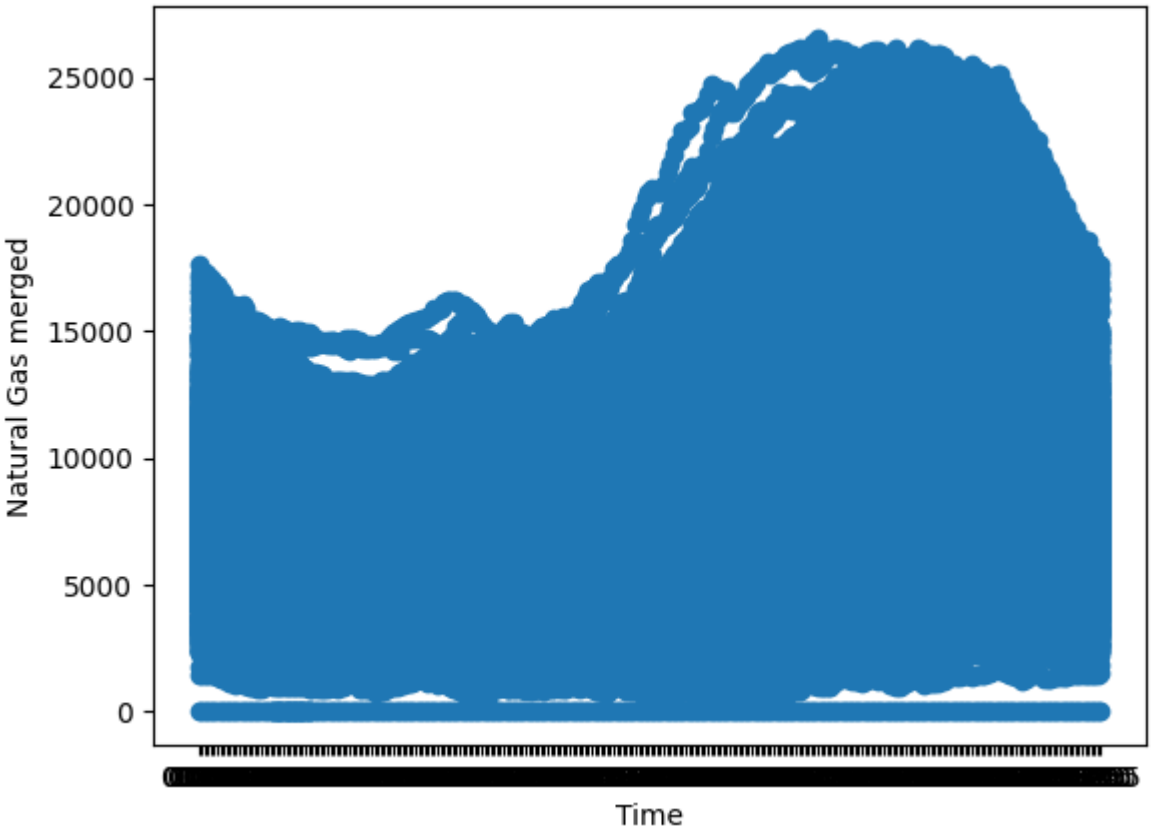

400000
300000
200000
100000
0
Small hydro
Time

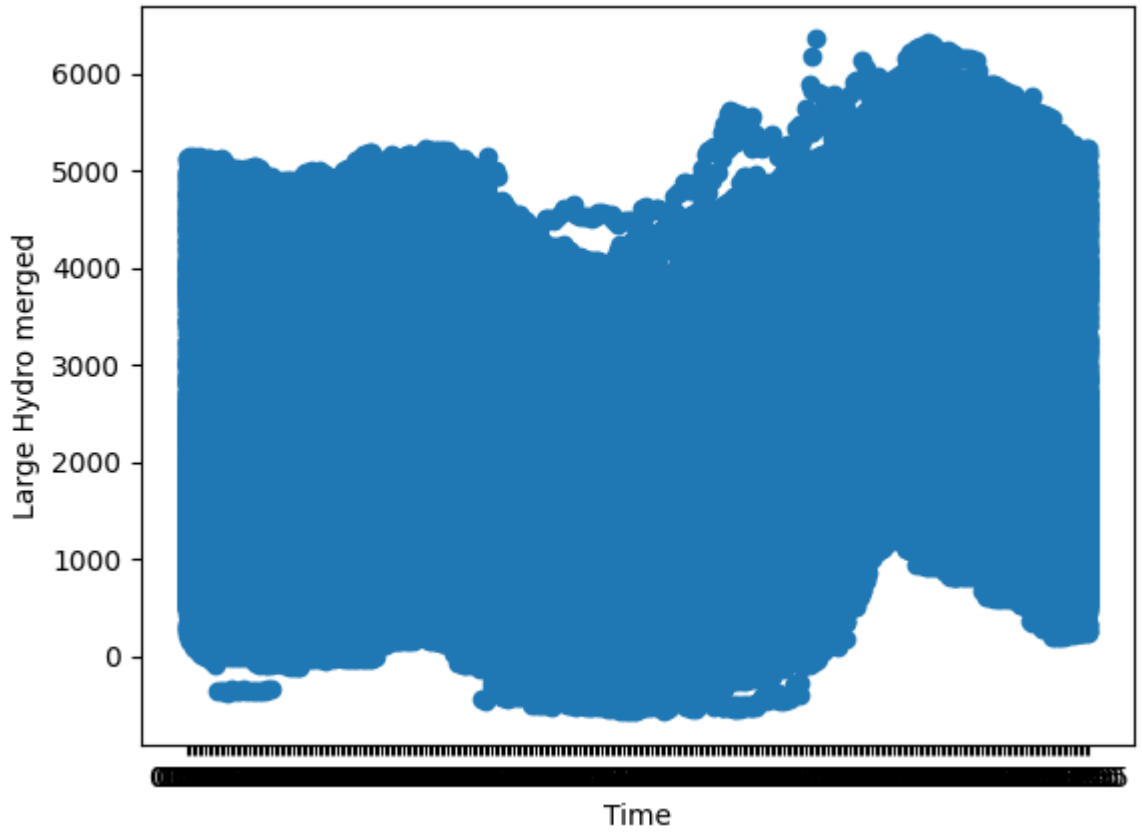

30
25
20
15
10
5
0
Coal
Time

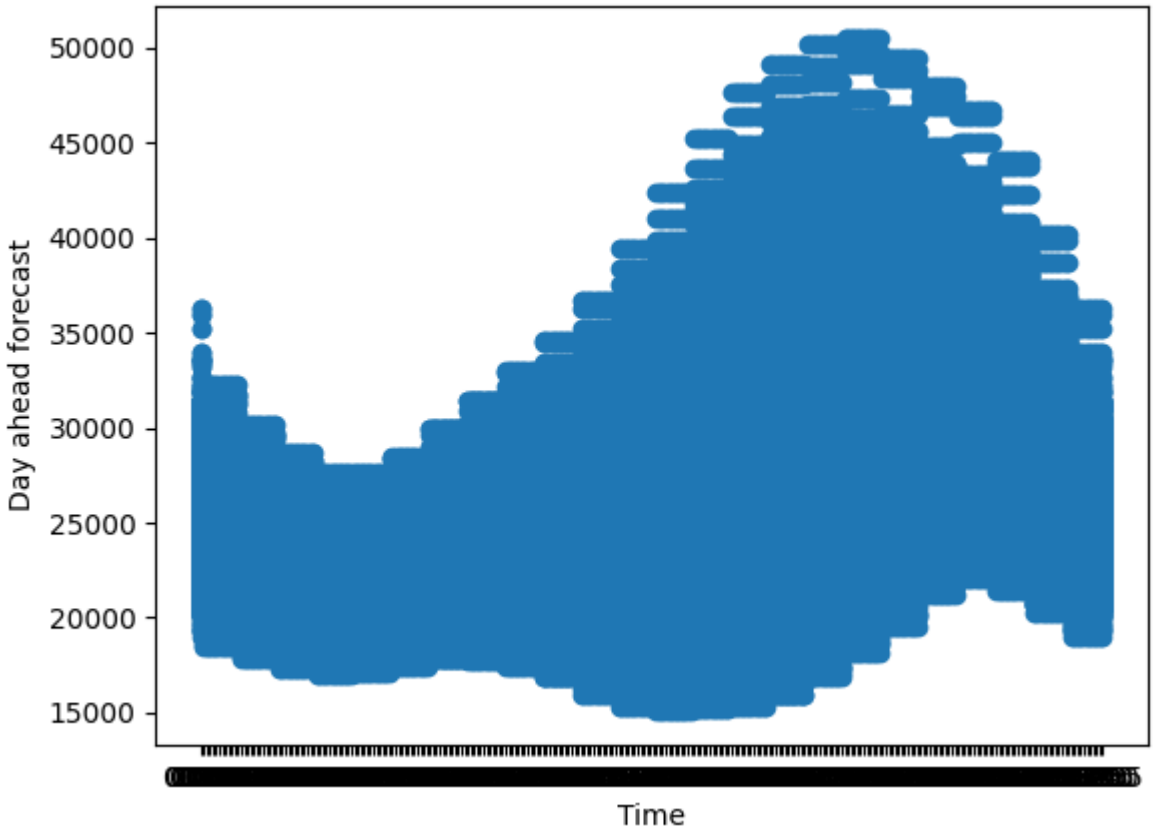

2000
1500
1000
500
0
Nuclear
Time

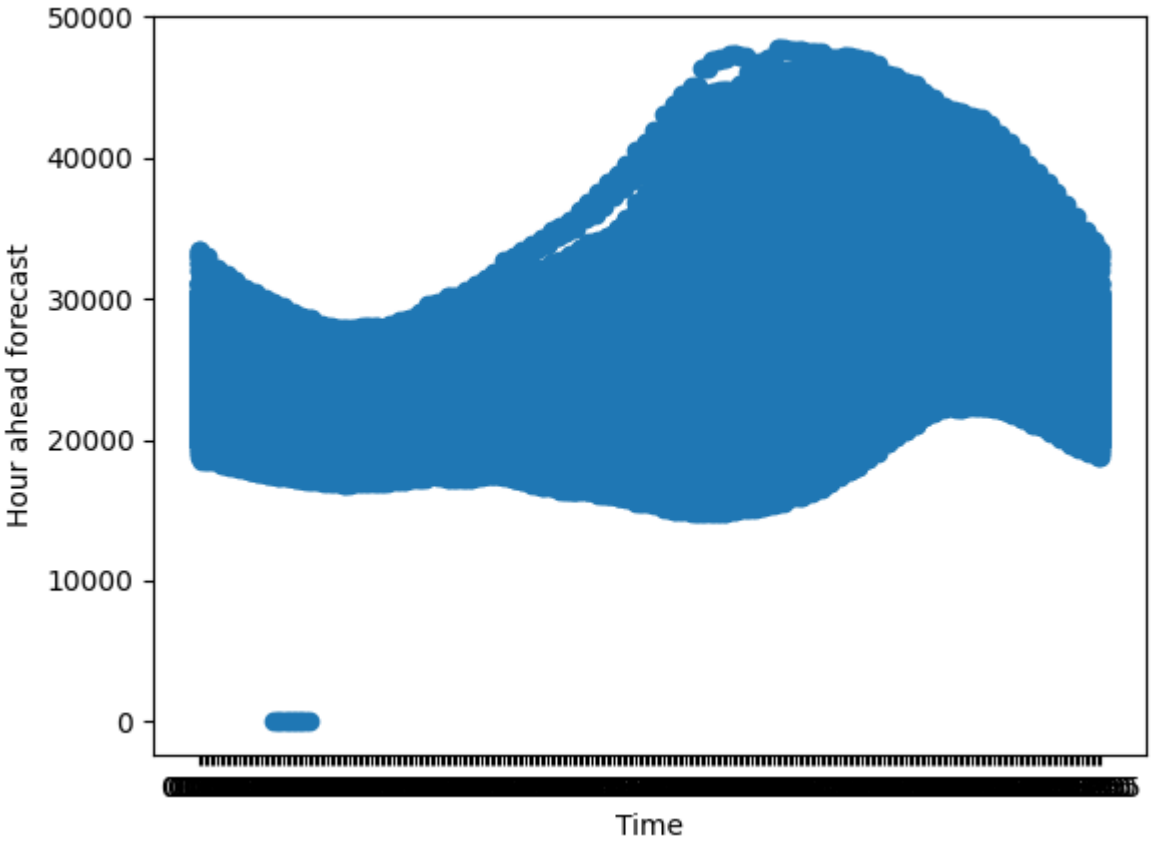

1500
1000
500
0
−500
−1000
−1500
Batteries
Time

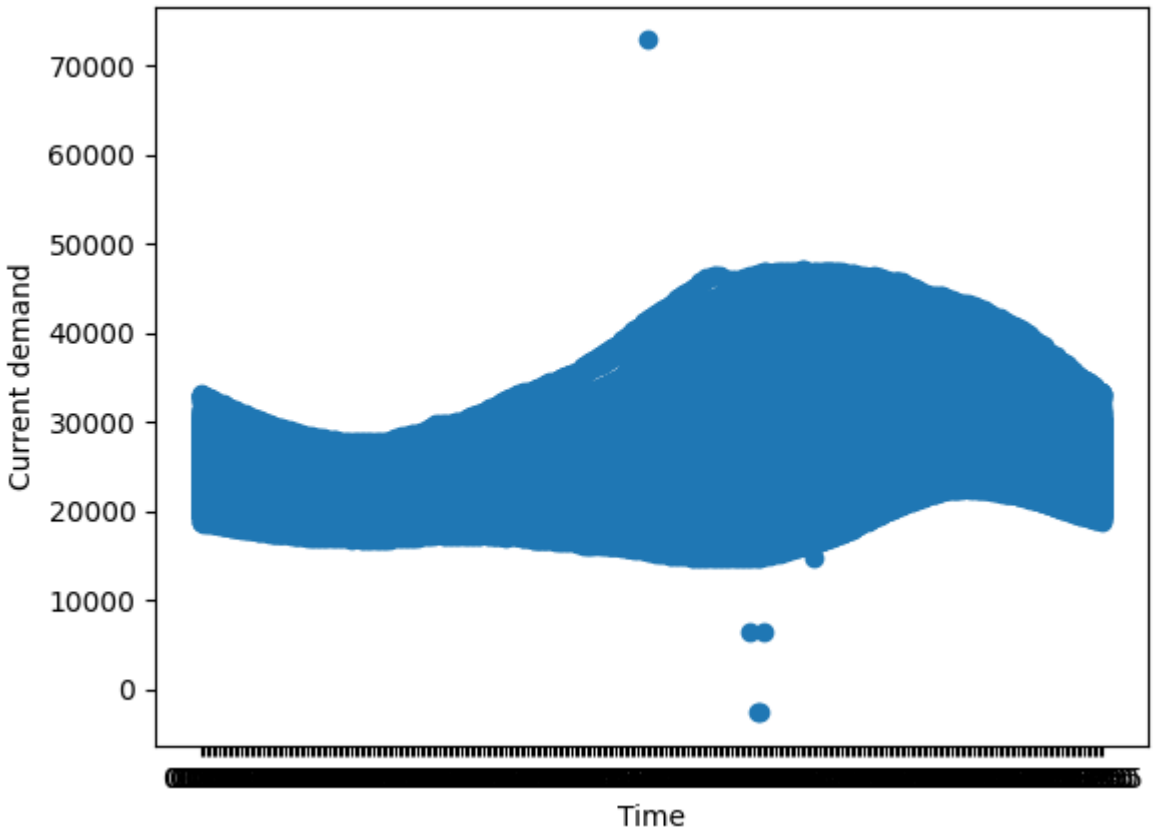

1.0
0.8
0.6
0.4
0.2
0.0
Other
Time


25000
20000
15000
10000
5000
0
Natural Gas merged
Time


Large Hydro merged
6000
5000
4000
3000
2000
1000
0
Time


Day ahead forecast
50000
45000
40000
35000
30000
25000
20000
15000
Time


50000
40000
30000
20000
10000
0
Hour ahead forecast
Time


70000
60000
50000
40000
30000
20000
10000
0
Current demand
Time

```python
# Basic metrics Section
def show_statistic_metrics(df):
    for column in df.columns:
        if column != "Time":
            print(column)
            print("mean:"+"\t"+str(df[column].mean()))
            print("deviation:"+"\t"+str(df[column].std()))
            print("median:"+"\t"+str(df[column].median()))
            print("mode:"+"\t"+str(df[column].mode()))
```

```
Solar
mean:    3580.3785809787532
deviation:       4357.152968795146
median: 125.0
mode:    0    0.0
```

```
Wind
mean:    2021.8344654749608
deviation:       1300.2753617383037
median: 1833.0
mode:    0    869.0
```

```
Geothermal
mean:    890.3561176835875
deviation:       115.93967779298323
median: 909.0
mode:    0    891.0
```

```
Biomass
mean:    311.2504863350928
deviation:       48.24635496659214
median: 315.0
mode:    0    338.0
```

```
Biogas
mean:    212.56730750952076
deviation:       15.383215114131437
median: 213.0
mode:    0    212.0
```

```
Small hydro
mean:   264.0541431947938
deviation:      778.0192614479195
median: 227.0
mode:   0    182.0
```

```
Coal
mean:   14.259094624650695
deviation:      4.523256444253206
median: 15.0
mode:   0    17.0
Name: Coal, dtype: float64
```

```
Nuclear
mean:   1867.4802725377504
deviation:      576.7221582621396
median: 2259.0
mode:   0    2266.0
```

```
Batteries
mean:   -5.02166488185384
deviation:      191.04237209469179
median: -3.0
mode:   0    0.0
```

```
Imports
mean:   6029.898861310538
deviation:      2613.5505314482807
median: 6468.0
mode:   0    2712.0
```

```
Other
mean:   0.0011247489116866165
deviation:      0.033518463734560855
median: 0.0
mode:   0    0.0
```

```
Natural Gas merged
mean:   8290.592283271973
deviation:      3922.9198540204297
median: 8000.0
```

```
Large Hydro merged
mean:   1897.3960415174922
deviation:      1288.9133654849225
median: 1551.0
mode:   0    938.0
```

```
Day ahead forecast
mean:   24954.443426477326
deviation:      4946.566113890026
median: 23954.0
mode:   0    23115.0
```

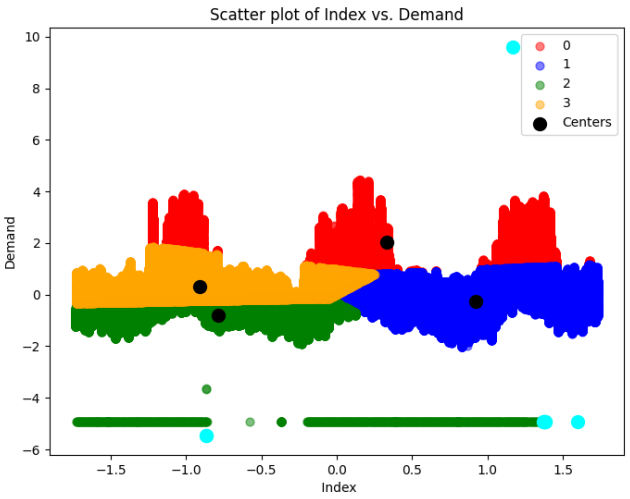
```
Hour ahead forecast
mean:   24836.803114979386
deviation:      4853.192348380815
median: 23887.0
mode:   0    23251.0
```

```
Current demand
mean:   24824.273172003097
deviation:      4862.37451748
median: 23867.0
mode:   0    19562.0
```

Ομαδοποίηση δεδομένων (Clustering)

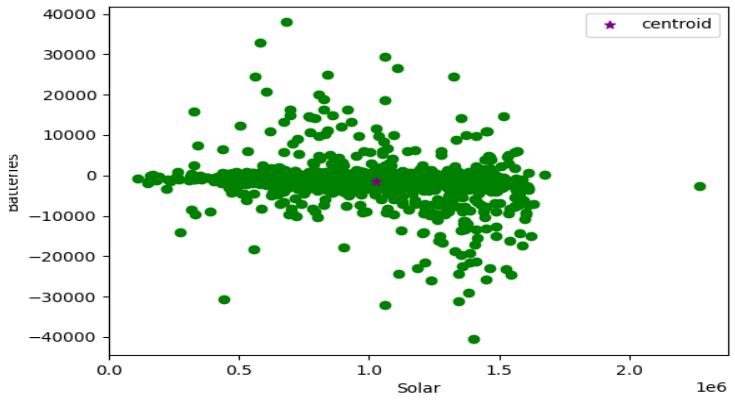
Στο δεύτερο ερώτημα της εργασίας κληθήκαμε να ομαδοποιήσουμε τις τιμές της ζήτησης και των πηγών για τον εντοπισμό απροσδόκητα υψηλών ή χαμηλών τιμών. Εφόσον ζητούνταν οι τιμές της ημέρας οι τιμές προστέθηκαν ανα 288 ωστέ να έχουμε τη συνολική κατανάλωση και παραγωγή της κάθε πηγής ημερησίως (Υπάρχουν 288 πεντάλεπτα ανά 24 ώρες)..Δυστυχώς δεν καταλάβαμε ως προς ποιο μέγεθος θα έπρεπε να συγκρίνουμε την εκάστοτε τιμή.Αν και εξετάσαμε την περίπτωση της χρήσης του αλγορίθμου DBSCAN λόγω της σταθερής πυκνότητας των δειγμάτων στον χρόνο αποφασίσαμε να υλοποιήσουμε την ομαδοποίηση μέσω της τεχνικής K-means. Λόγω του προβλήματος που αναφέρθηκε υλοποιήθηκαν οι εξής σκέψεις.

1) Ομαδοποίηση των τιμών ως προς το index του ενιαίου αρχείου. Μας δίνει μια ομαδοποίηση τόσο στον χρόνο όσο και στο πεδίο των τιμών.



Ενδεικτικά, η ζήτηση ως προς το index, διαμερισμένη σε 3 clusters. Τα outliers εμφανίζονται με γαλάζιο χρώμα

2) Ομαδοποίηση των τιμών κάποιας πηγής ώς προς άλλη πηγή ή ως προς την τρέχουσα ζήτηση. Αδυνατούμε ωστόσο να εξάγουμε κάποια λογική συσχέτιση από τα παραγόμενα γραφήματα.



Ενδεικτικά, η συσχέτιση ηλιακής ενέργειας και ενέργειας παραγόμενης από μπαταρίες.

Ελπίζουμε πως ακόμα και αν καμία από τις παραπάνω ιδέες δεν είναι έγκυρες, να αξιολογηθεί η υλοποίηση ομαδοποιήσεων εν γένει.

Αναλυτικά:

Για τις δύο αυτές προσεγγίσεις, κάθε φορά προσαρμόσαμε τα δεδομένα κατάλληλα στους έξονες με τις συναρτήσεις StandardScaler και MinManScaler αντίστοιχα.Εφόσον χρησιμοποιήσαμε τον αλγόριθμο Kmeans, για να καλύψουμε το ενδεχόμενο του να χρειάζεται να τρέξουμε kmeans για όλες τις στήλες που σημαίνει ότι δε θα μπορούσαμε να αναπαριστούμε κάθε φορά γραφικά τα δεδομένα και να αριθμούμε τα clusters, αναφέρεται η βιβλιοθήκη KneeLocator η οποία προγραμματιστικά υπολογίζει που κάνει την μέγιστη

καμπύλη η elbow plot. Για τα δεδομένα της άσκησης όπου έχουμε συσχετίσει μόνο δύο πηγές, υπολογίζουμε το πλήθος των clusters με το μάτι.

Συνένωση των rows και άθροισμα ανά ημέρα:

```
for column in df.columns:
    for day in range(0, 1096):
        per_day = df.loc[day * step:(day + 1) * step - 1, column].sum()
        per_day_values.append(per_day)

    per_day_df[column] = per_day_values
    temp_list.clear()
    per_day_values.clear()
```

Kmeans & elbow method programmatically:

```
for k in k_rng:
    km = KMeans(n_clusters=k)
    km.fit(per_day_df[['Solar','Batteries']])
    sse.append(km.inertia_)
```

Ευκλείδεια απόσταση από το κέντρο των clusters για τον υπολογισμό των outliers:

```
def distance_from_center(Index, Demand, label):
    center_Index =  model.cluster_centers_[label,0]
    center_Demand =  model.cluster_centers_[label,1]
    distance = np.sqrt((Index - center_Index) ** 2 + (Demand - center_Demand) ** 2)
    return np.round(distance, 3)
df['label'] = model.labels_
df['distance'] = distance_from_center(df.Index, df.Demand, df.label)
```

Εκπαίδευση νευρωνικού δικτύου LSTM και πρόβλεψη μελλοντικών τιμών

Όσον αφορά την πρόβλεψη μελλοντικών τιμών ζήτησης για τις μη ανανεώσιμες πηγές ενέργειας, χρησιμοποιήσαμε τα προηγούμενα στοιχεία ως σημείο αναφοράς. Πιο συγκεκριμένα μη ανανεώσιμες πηγές ενέργειας στα δεδομένα μας είναι ο άνθρακας (Coal) καθώς και το φυσικό αέριο το οποίο όπως προαναφέρθηκε παραπάνω έχει τις τιμές του στην στήλη Natural Gas merged. Η πρόσθεση των δύο ανωτέρω στηλών μας δίνει τα ιστορικά δεδομένα για τις ανάγκες ενέργειας της Καλιφόρνια από μη ανανεώσιμες πηγές.

Χρησιμοποιήθηκε η βιβλιοθήκη tensorflow με το keras ως διεπαφή για την δημιουργία και εκπαίδευση νευρωνικού δικτύου. Το νευρωνικό δίκτυο μελετά το 70% του πλήθους των τιμών σε 6-άδες όπου χρησιμοποιώντας τις 5 πρώτες επιχειρεί να εκτιμήσει την έκτη τιμή. Η διαδικασία αυτή επαναλαμβάνεται για το σύνολο των δεδομένων σε 20 epochs. Στην συνέχεια για τον έλεγχο ακρίβειας εισάγεται τυχαία 5-άδα δεδομένων από το υπολειπόμενο 30% των δεδομένων και το δίκτυο εκτυπώνει την πρόβλεψη του καθώς και τα εισαγόμενα δεδομένα. Από τις παρατηρήσεις μας θεωρούμε ότι η ακρίβεια του δικτύου αν και όχι απόλυτη είναι ικανοποιητική συναρτήσει του χρόνου για την εκτέλεση του προγράμματος. Γνωρίζουμε πως αν αυξάναμε τον αριθμό τον epochs, το πλήθος των δεδομένων εκπαίδευσης αλλά και ενδεχομένως το πλήθος Ν στις Ν-αδες εισαγωγής δεδομένων θα είχαμε ενδεχομένως καλύτερες επιδόσεις. Για τον εκπαιδευτικό σκοπό του μαθήματος και με τα μέσα που διαθέτουμε θεωρούμε το αποτέλεσμα ικανοποιητικό.

Ακολουθούν ενδεικτικές εκτυπώσεις του δικτύου.

```
Program predicted:
[[9446.451]]
Actual Value was:
[9435.]
```

```
Program predicted:
[[15672.347]]
Actual Value was:
[15358.]
```

```
Program predicted:
[[15650.431]]
Actual Value was:
[15627.]
```

Για την αξιολόγηση του προβλεπτή χρησιμοποιήσαμε την μέθοδο holdout για Κ αποτελέσματα με μέση απόκλιση 0.2% σε εύρος 0.1-0.3%. Οι αποκλίσεις προκύπτουν αφαιρώντας την πραγματική τιμή από την προβλεπόμενη και διαιρώντας το αποτέλεσμα με την πραγματική τιμή. Τέλος ως απόκλιση θεωρείται το πηλίκο της απόλυτης τιμής.

Παρατήρηση

Λόγω προγραμματιστικού λάθους στις πρώτες εκτελέσεις του προγράμματος το σύστημα εκπαιδευόταν στις πρώτες 80 τιμές δεδομένων και εξεταζόταν στις υπόλοιπες. Αυτό είχε ως αποτέλεσμα σαφώς υποδεέστερη ακρίβεια αλλά τάχιστη εκπαίδευση και παραγωγή αποτελεσμάτων. Ενδεικτικά, το σύστημα εκπαιδεύτηκε σε 10.000 epochs εντός χρόνου μικρότερου των 3 λεπτών και απέδωσε αποτέλεσμα με απόκλιση της τάξης του 0.2 - 5%. Αν και δεν είμαστε σε θέση να γνωρίζουμε τα αποδεκτά επίπεδα ακρίβειας θεωρούμε πως ένα υπεργρήγορο σύστημα που εκπαιδεύεται σε μικρό χρόνο και παράγει σχετικά έγκυρα αποτελέσματα είναι άξιο αναφοράς. Ενδεχομένως να οφείλεται στην μορφή των δεδομένων με τα επαναλαμβανόμενα μοτίβα ή στην πυκνότητα των δεδομένων στο αρχικό dataset. Θα συμπεριληφθεί και αυτή η “ελαττωματική” υλοποίηση στην τελική παράδοση.

```
Epoch 3000/3000
3/3 [==============================] - 0s 2ms/step - loss: 4656.4517 - accuracy: 0.0000e+00
1/1 [==============================] - 0s 138ms/step
Program predicted:
[[14309.211]]
Actual Value was:
[14277.]
```

```
Epoch 3000/3000
3/3 [==============================] - 0s 2ms/step - loss: 7039.5952 - accuracy: 0.0000e+00
1/1 [==============================] - 0s 129ms/step
Program predicted:
[[8452.695]]
Actual Value was:
[8402.]
```

Ενδεικτικά αποτελέσματα σε 3000 epochs. Απόκλιση της τάξης του 0.2-0.5%

Hold-out μέθοδος 10 επαναλήψεων, μέση απόκλιση 2%

Πρόβλεψη βαθμολογίας με βάση την κριτική κειμένων χρησιμοποιώντας Word Embeddings και Random Forest

Πρώτη φάση: Word Embeddings.

Για την μετατροπή των κειμένων σε διανύσματα, ακολουθήσαμε τα εξής βήματα: Πρώτα, περάσαμε το csv αρχείο σε ένα dataframe. Το dataframe αυτό το μοιράσαμε σε δύο τμήματα σε train size και test size. Επειδή όμως το πλήθος των κριτικών με βαθμολογία μικρότερη του 5 ήταν πολύ μικρό και δεν πετυχαίναμε καλά αποτελέσματα αποδόσεων,

```
percentage of 1 score is 0.10042% Of the dataset
percentage of 2 score is 0.05836% Of the dataset
percentage of 3 score is 0.07804% Of the dataset
percentage of 4 score is 0.16768% Of the dataset
percentage of 5 score is 0.5955% Of the dataset
```

πήραμε κατεπανάληψη, όλες τις κριτικές ανά βαθμολογία και ως τελικό training και data set πήραμε το 0.8 και 0.2 από την κάθε βαθμολογία αντίστοιχα.

```python
def get_sets(df):

    train_df = pd.DataFrame()
    test_df = pd.DataFrame()

    for score in range(1, 6):
     helper_df = df[df.Score==score] #filto gia ka8e va8mologia
     train_size = int(helper_df.shape[0] * 0.8)
     df_train_score = helper_df.loc[:train_size,:]
     df_test_score = helper_df.loc[train_size:,:]
     train_df = pd.concat([train_df,df_train_score],axis=0)
     test_df = pd.concat([test_df,df_test_score],axis=0)

    print(train_df)
    return train_df,test_df
```

Στη συνέχεια, έπρεπε να “καθαρίσουμε” τις κριτικές που περιέχονταν στο κάθε set ώστε να τα περάσουμε από tokenization και να πάρουμε τα τελικά μας διανύσματα. Το καθάρισμα που έπρεπε να γίνει ήταν: να αφαιρεθούν κάποιες συντομογραφίες του τύπου “can't” αντί για “can not”,

```
CONTRACTIONS_DICT = {"can`t":"can not",
                     "won`t":"will not",
                     "don`t":"do not",
                     "aren`t":"are not",
                     "i`d":"i would",
                     "couldn`t": "could not",
                     "shouldn`t": "should not",
                     "wouldn`t": "would not",
                     "isn`t": "is not",
                     "it`s": "it is",
                     "didn`t": "did not",
                     "weren`t": "were not",
                     "mustn`t": "must not",
                    }
```

```
def replace_words(string:str, dictionary:dict):
    for shortcut, full_word in dictionary.items():
        string = string.replace(shortcut, full_word)
    return string
```

```
def prepare_data(df):

    df["text"] = df["Text"] \
                .apply(lambda x: replace_words(x,CONTRACTIONS_DICT)) \
                .apply(lambda x: clean_text(x))

    return df.text.values, df.Score.values
```

να αφαιρεθούν τα σημεία στίξης, ta url-html στοιχεία, σε όλες τις λέξεις να εφαρμοστεί μικρή γραμματοσειρά και να χωριστούν μία-μία οι λέξεις κάθε κριτικής.

```python
def clean_text(
    string: str,
    punctuations=r'''!()-[]{};:'"\,<>./?@#$%^&*_~''',
    stop_words=['the', 'a', 'and', 'is', 'be', 'will',]) -> str:

    #Cleaning the urls
    string = re.sub(r'https?://\S+|www\.\S+', '', string)

    #Cleaning the html elements
    string = re.sub(r'<.*?>', '', string)

    #Removing the punctuations
    for x in string.lower():
        if x in punctuations:
            string = string.replace(x, "")

    #Converting the text to lower
    string = string.lower()

    #Removing stop words
    string = ' '.join([word for word in string.split() if word not in stop_words])

    return string
```

Εφαρμογή:

```python
df = pd.read_csv('amazon.csv')

training_df, testing_df = get_sets(df)

#clean the text
train_reviews, train_scores= prepare_data(training_df)
test_reviews, test_scores = prepare_data(testing_df)
```

Μέχρι στιγμής, έχουμε ‘μαζέψει’ όλες τις κριτικές μία προς μία σε λίστες train_reviews,train_scores και ούτω καθεξής. Για την μετατροπή αυτών των κριτικών σε διανύσματα, χρησιμοποιήσαμε την κλάση Tokenizer της βιβλιοθήκης `keras.preprocessing.text` η οποία, με την κλήση της fit_on_texts δημιουργεί ένα λεξικό με βάση το input δηλαδή, το training set. Κάθε κριτική αντιστοιχίζεται σε μία γραμμή διανύσματος με μία τιμή για κάθε λέξη της.(μέθοδος text_to_matrix)

Γιατί tfidf ανάμεσα στα modes του keras vectorizer

Για ένα πολύ μικρό- ενδεικτικό σύνολο δεδομένων (500 dataframe rows), εκτελέσαμε το ίδιο πρόγραμμα και είδαμε πως χρησιμοποιώντας τη μέθοδο tf-idf πετυχαίναμε καλύτερες αξιολογήσεις σε σχέση με τα υπόλοιπα word embeddings μοντέλα όπως φαίνεται, με χαρακτηριστικό το παράδειγμα για το score=4 όπου μόνο h μέθοδος tfidf έχει πετύχει έστω και ελάχιστες φορές το σωστό αποτέλεσμα αφού εξ’ορισμού, ποσοτικοποιεί και διατηρεί την

“σημασία” και για τα δεδομένα που δεν εμφανίζονται τόσο συχνά σε αντίθεση με την freq, ή η count κ.ο.κ..

	precision	recall	f1-score	support
1	0.00	0.00	0.00	58
2	0.00	0.00	0.00	24
3	0.00	0.00	0.00	34
4	0.25	0.01	0.02	97
5	0.57	0.99	0.73	287
accuracy			0.57	500
macro avg	0.16	0.20	0.15	500
weighted avg	0.38	0.57	0.42	500

Count

	precision	recall	f1-score	support
1	0.00	0.00	0.00	58
2	0.00	0.00	0.00	24
3	0.00	0.00	0.00	34
4	0.00	0.00	0.00	97
5	0.58	1.00	0.73	287
accuracy			0.57	500
macro avg	0.12	0.20	0.15	500
weighted avg	0.33	0.57	0.42	500

Binary

	precision	recall	f1-score	support
1	0.00	0.00	0.00	58
2	0.00	0.00	0.00	24
3	0.00	0.00	0.00	34
4	0.00	0.00	0.00	97
5	0.57	1.00	0.73	287
accuracy			0.57	500
macro avg	0.11	0.20	0.15	500
weighted avg	0.33	0.57	0.42	500

Freq

	precision	recall	f1-score	support
1	0.00	0.00	0.00	58
2	0.00	0.00	0.00	24
3	0.00	0.00	0.00	34
4	0.00	0.00	0.00	97
5	0.58	1.00	0.73	287
accuracy			0.57	500
macro avg	0.12	0.20	0.15	500
weighted avg	0.33	0.57	0.42	500

Γιατί td idf σε σχέση με Word2Vec και GloVe:

Η Word2Vec έχει ελαφρώς χαμηλότερη απόδοση από την tdidf(Με βάση της ιστοσελίδα geeks for geeks) και επιπλέον, αν και η Word2Vec προτείνεται σε ανάλυση κειμένων και τη συσχέτιση λέξεων μεταξύ τους, στη συγκεκριμένη περίπτωση που έχουμε κλίμακες βαθμών με μικρή απόκλιση, βολεύει περισσότερο η Tf Idf αφού ανιχνεύει καλύτερα τα keywords και δίνει μεγάλη σημασία και στις λέξεις που εμφανίζονται σπάνια (χρήσιμο για εμάς αφού έχουμε 10% κακές και 60% καλές κριτικές).

Σε σχέση με την GloVe καλύτερη σε προβλήματα που αναζητούμε την σχέση λέξεων μεταξύ τους και όχι λέξεις κλειδιά για την εξαγωγή συμπεράσματος βαθμών όπως στην περίπτωσή μας.

```
#mporouse na ginei ki egw h epejergasia twn texts alla gia
#sugkekrimena xarakthristika
tokenizer = Tokenizer()
tokenizer.fit_on_texts(train_reviews)

#dunata modes: binary, count, tfidf,freq
train_tokenized = tokenizer.texts_to_matrix(
                                train_reviews,
                                mode='tfidf'
                                )
test_tokenized = tokenizer.texts_to_matrix(
                                test_reviews,
                                mode='tfidf'
                                )
```

Για τον Random forest classifier, χρησιμοποιήσαμε την αντίστοιχη κλάση της βιβλιοθήκης Sklearn, με παραμέτρους: n_estimators=200 (default είναι 100 αλλά δεν πετυχαίναμε καλές αποδόσεις οπότε και διπλασιάσαμε το πλήθος των decision trees), n_jobs=-1 δηλαδή όλοι οι πυρήνες του συστήματος να αξιοποιηθούν,oob_score = True (μετρική για την αύξηση της απόδοσης)

```
forest_classifier = RandomForestClassifier(
                                    n_estimators=200,
                                    oob_score=True,
                                    n_jobs=-1,
                                    )
forest_classifier.fit(train_tokenized,train_scores)

predictions = forest_classifier.predict(test_tokenized)
```

Και τέλος, οι μετρικές f1 score, precision και recall από την βιβλιοθήκη sklearn.metrics

```
print(confusion_matrix(test_scores,predictions))
print(classification_report(test_scores,predictions))
print(accuracy_score(test_scores,predictions))
```

Δυστυχώς, το πλήθος των δεδομένων δεν αρκεί και γι'αυτό τα αποτελέσματα για τις τιμές πέραν του 5 που έχουν όλες μαζί 40% του dataset δεν επιτυγχάνει την πρόβλεψη.

Για το μισό dataset:

	precision	recall	f1-score	support
1	0.00	0.00	0.00	64
2	0.00	0.00	0.00	21
3	0.00	0.00	0.00	40
4	1.00	0.01	0.01	145
5	0.87	1.00	0.93	1757

Για ολόκληρο το dataset:

	precision	recall	f1-score	support
1	0.00	0.00	0.00	4546
2	0.00	0.00	0.00	2798
3	0.00	0.00	0.00	3637
4	0.00	0.00	0.00	7166
5	0.47	1.00	0.64	15963